Analytics Engineering: Session 7

Jinja, Macros & Packages

Agenda

- dbt Packages & dbt Hub
- Jinja Templating
- Creating Custom Macros
- Using dbt-utils & dbt-expectations
- Git Workflow in dbt Development

dbt Packages

Libraries of dbt code you can import. Defined in `packages.yml`:

```yaml
packages:
  - package: dbt-labs/dbt_utils
    version: 1.1.1
  - package: dbt-labs/codegen
    version: 0.12.1
  - package: calogica/dbt_expectations
    version: 0.10.1
```

Install with: `dbt deps`

Browse packages at hub.getdbt.com.

Key Packages We Use

Package	Purpose
dbt-utils	Essential SQL utilities (surrogate keys, unions, pivots)
codegen	Generate YAML and SQL boilerplate from sources
dbt-expectations	Advanced data quality tests (Great Expectations-style)

Jinja Templating

The language that makes SQL dynamic.

- **Variables:** `{{ my_var }}`
- **Control Flow:** `{% if %}`, `{% for %}`
- **Functions:** `{{ ref() }}`, `{{ source() }}`

```
select
{% for col in ['a', 'b', 'c'] %}
    sum({{ col }}) as sum_{{ col }}{% if not loop.last %},{% endif %}
{% endfor %}
from {{ ref('my_model') }}
```

Jinja Control Flow

```
-- Conditional logic
select *
from {{ ref('stg_orders') }}
{% if target.name == 'dev' %}
  where order_date > '2024-01-01'  -- limit data in dev
{% endif %}
```

```
-- Variables
{% set payment_methods = ['credit_card', 'bank_transfer', 'cash'] %}

select
{% for method in payment_methods %}
    sum(case when payment_method = '{{ method }}' then amount end) as {{ method }}_total
    {% if not loop.last %},{% endif %}
{% endfor %}
```

Creating Custom Macros

Reusable functions defined in the `macros/` directory.

```
-- macros/cents_to_dollars.sql
{% macro cents_to_dollars(column_name) %}
    ({{ column_name }} / 100)::numeric(16, 2)
{% endmacro %}
```

Usage in a model:

```
select
    order_id,
    {{ cents_to_dollars('amount_cents') }} as amount_dollars
from {{ ref('stg_payments') }}
```

Using dbt-utils

`generate_surrogate_key()` — Create a unique key from multiple columns:

```
select
    {{ dbt_utils.generate_surrogate_key(['order_id', 'product_id']) }} as order_item_key,
    order_id,
    product_id
from {{ ref('stg_order_items') }}
```

`union_relations()` — Union multiple tables with the same schema:

```
{{ dbt_utils.union_relations(
    relations=[ref('orders_2023'), ref('orders_2024')]
) }}
```

Using dbt-expectations

Advanced data quality tests defined in YAML:

```yaml
models:
  - name: mart_orders
    columns:
      - name: total_amount
        tests:
          - dbt_expectations.expect_column_values_to_be_between:
              min_value: 0
              max_value: 100000
      - name: order_date
        tests:
          - dbt_expectations.expect_column_values_to_be_of_type:
              column_type: date
```


dbt-expectations brings Great Expectations-style tests to dbt.

Git Workflow in dbt

The certification exam tests git concepts within the dbt development lifecycle.

1. **Branch**: Create a feature branch for your work.

```
git checkout -b feature/add-revenue-model
```

2. **Commit**: Save changes with meaningful messages.
3. **Push**: Send to remote repository.
4. **Pull Request**: Request review from teammates.
5. **Merge**: Integrate into main branch after approval.

Atomic commits — one logical change per commit.

What have we learned in this session

- Install and use dbt packages (dbt-utils, dbt-expectations, codegen)
- Write Jinja templates with variables, loops, and conditionals
- Create custom macros for reusable logic
- Use `generate_surrogate_key()` and `union_relations()`
- Practice the git workflow: branch, commit, push, PR

Next Session: Practice Session II: Advanced Modeling & Refactoring.